

Python für EPROG

Cheatsheet · Serie 1 bis 5

Alles was du für die ersten fünf Serien der Lehrveranstaltung **101.275 Einführung in das Programmieren für Technische Mathematik** (SS 2026) brauchst — mit Erklärungen, Beispielen und typischen Stolperfallen.

Inhalt

- Kapitel 1 · Grundlagen: Werte, Variablen, Ein- und Ausgabe
- Kapitel 2 · Typen & Operatoren: Zahlen, Strings, Booleans
- Kapitel 3 · Kontrollfluss: if / elif / else
- Kapitel 4 · Schleifen: for, while, range, break, continue
- Kapitel 5 · Listen & Strings: Indexing, Slicing, Methoden
- Kapitel 6 · Funktionen: def, return, Parameter
- Kapitel 7 · Module: math, random
- Kapitel 8 · Typische Patterns aus EPROG Serien 1–5

● Tipp

Dieses PDF ist zum **Nachschlagen** gedacht, nicht zum Linearlesen. Geh direkt zum Kapitel, das du gerade brauchst. Jedes Codebeispiel ist so klein wie möglich und so groß wie nötig.

1 · Grundlagen

Ein Python-Programm ist eine Folge von Anweisungen, die von oben nach unten ausgeführt werden. Das absolute Minimum, das du immer brauchst:

1.1 · Ausgabe mit print()

print() schreibt etwas in die Konsole. Mehrere Dinge werden mit Komma getrennt — Python fügt automatisch ein Leerzeichen ein.

```
print("Hallo Welt")
print("Ergebnis:", 42)
print(3 + 4)
```

Ausgabe:

```
Hallo Welt
Ergebnis: 42
7
```

1.2 · Eingabe mit input()

input() liest eine Zeile, die der Benutzer tippt. **Sehr wichtig:** **input()** liefert *immer* einen String zurück — auch wenn der Benutzer eine Zahl eintippt.

```
name = input("Wie heißt du? ")
print("Hi", name)
```

● Achtung

Wenn du mit Zahlen rechnen willst, musst du den String **umwandeln**:

```
alter_str = input("Alter: ")      # "25"   (String!)
alter      = int(alter_str)        # 25     (Integer)
print("In 10 Jahren:", alter + 10)
```

1.3 · Variablen & Zuweisung

Mit **=** weist du einen Wert einer Variablen zu. Python "lernt" dabei automatisch den Typ.

```
x = 5                # int
pi = 3.14            # float
name = "Hani"        # str
fertig = True        # bool
```

● Tipp

Variablennamen: Kleinbuchstaben, Wörter mit Unterstrich trennen (**snake_case**). Nie mit einer Zahl beginnen, keine Leerzeichen, keine Umlaute.

1.4 · Kommentare

```
# Das ist ein einzeiliger Kommentar  
x = 5  # auch hinter Code
```

2 · Typen & Operatoren

2.1 · Die vier wichtigsten Typen

Typ	Beispiel	Bedeutung
int	0, 42, -7	Ganze Zahlen
float	3.14, -0.5, 2.0	Kommazahlen
str	"Hallo", 'Hi'	Text (String)
bool	True, False	Wahrheitswert

Mit **type(x)** kannst du jederzeit prüfen, welcher Typ in einer Variable steckt:

```
type(42)      # <class 'int'>
type("42")    # <class 'str'>
type(3.14)    # <class 'float'>
```

2.2 · Typ-Umwandlung (Casting)

```
int("7")      # 7      (String -> int)
float("3.14")  # 3.14   (String -> float)
str(42)        # "42"   (int -> String)
int(3.9)       # 3      (Nachkommastellen werden abgeschnitten, nicht gerundet!)
```

● Achtung

int(3.9) ergibt **3**, nicht 4. Python rundet nicht, sondern schneidet ab. Zum Runden:
round(3.9) → 4.

2.3 · Arithmetische Operatoren

Operator	Bedeutung	Beispiel	Ergebnis
+	Addition	5 + 2	7
-	Subtraktion	5 - 2	3
*	Multiplikation	5 * 2	10
/	Division (float)	5 / 2	2.5
//	Ganzzahl-Division	5 // 2	2
%	Modulo (Rest)	5 % 2	1

**	Potenz	2 ** 10	1024
-----------	--------	---------	------

● Tipp

Modulo (%) ist dein bester Freund. **a % b = 0** bedeutet: a ist durch b teilbar. So testest du gerade Zahlen, Vielfache, etc.

```
x % 2 == 0          # True wenn x gerade
jahr % 4 == 0       # durch 4 teilbar (Schaltjahr-Check)
n % 15 == 0         # durch 15 teilbar (FizzBuzz!)
```

2.4 · Vergleichs- und Logik-Operatoren

Vergleiche liefern immer einen Booleschen Wert (True oder False):

```
==   gleich          !=   ungleich
<    kleiner         >    größer
<=   kleiner-gleich >=   größer-gleich
```

● Achtung

= ist Zuweisung. == ist Vergleich. Das ist der häufigste Anfängerfehler überhaupt.

Logik verknüpft Booleans:

```
and   # beide wahr
or    # mindestens eines wahr
not   # Verneinung

x > 0 and x < 10      # x zwischen 0 und 10
0 < x < 10            # Python erlaubt auch das (gleichwertig)
not (x == 0)         # x ist nicht 0
```

3 · Kontrollfluss: if / elif / else

Mit **if** steuerst du, ob ein Codeblock ausgeführt wird. In Python ist die **Einrückung** (Indent) Teil der Syntax — alles, was zu einem if gehört, wird **4 Leerzeichen** eingerückt.

3.1 · Basis-Struktur

```
if bedingung:
    # Code, wenn wahr
elif andere_bedingung:
    # Code, wenn die auch wahr ist
else:
    # Code, wenn nichts zutraf
```

elif und **else** sind beide optional. Nach dem Doppelpunkt **muss** ein Zeilenumbruch und Einrückung kommen.

3.2 · Konkretes Beispiel

```
note = int(input("Note: "))

if note == 1:
    print("Sehr gut")
elif note <= 3:
    print("Bestanden")
elif note == 4:
    print("Gerade noch")
else:
    print("Nicht bestanden")
```

3.3 · FizzBuzz — das Klassiker-Pattern

Aus Serie 3. Die Reihenfolge ist entscheidend: **erst** auf 15 prüfen, dann auf 3 und 5. Sonst würdest du "Fizz" ausgeben, bevor du überhaupt zum FizzBuzz-Check kommst.

```
for i in range(1, 21):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

3.4 · Schaltjahr-Regel (Serie 3)

Ein Jahr ist ein Schaltjahr, wenn:

- es durch 4 teilbar ist,
- **außer** es ist durch 100 teilbar,
- **aber** doch wenn es durch 400 teilbar ist.

```
if jahr % 400 == 0:  
    schaltjahr = True  
elif jahr % 100 == 0:  
    schaltjahr = False  
elif jahr % 4 == 0:  
    schaltjahr = True  
else:  
    schaltjahr = False
```

● **Tipp**

Kürzer mit Logik:

```
schaltjahr = (jahr % 4 == 0 and jahr % 100 != 0) or (jahr % 400 == 0)
```

4 · Schleifen

Eine Schleife wiederholt Code. Python hat zwei Arten: **for** (wenn du weißt, wie oft) und **while** (wenn du bis zu einer Bedingung wiederholst).

4.1 · for-Schleife mit range()

range() erzeugt eine Folge von ganzen Zahlen. Drei Varianten:

```
range(5)           # 0, 1, 2, 3, 4      (Stop ist exklusiv!)
range(2, 7)        # 2, 3, 4, 5, 6      (Start, Stop)
range(0, 10, 2)     # 0, 2, 4, 6, 8      (Start, Stop, Schritt)
range(10, 0, -1)    # 10, 9, 8, ..., 1   (rückwärts)
```

● Achtung

Die **obere Grenze** ist in Python **immer exklusiv**. `range(1, 10)` geht von 1 bis 9, nicht bis 10.

Anwendung:

```
for i in range(1, 6):
    print(i, "Quadrat:", i ** 2)

# Summe 1 + 2 + ... + 100
summe = 0
for i in range(1, 101):
    summe = summe + i
print(summe)  # 5050
```

4.2 · for-Schleife über Listen und Strings

```
for zahl in [3, 1, 4, 1, 5, 9]:
    print(zahl)

for buchstabe in "Python":
    print(buchstabe)
```

4.3 · while-Schleife

Wiederhole, solange eine Bedingung wahr ist. Die Bedingung wird **vor jedem** Durchlauf geprüft.

```
n = 1
while n < 100:
    n = n * 2
print(n)  # 128 (erste Zweierpotenz ≥ 100)
```


● Achtung

Pass auf, dass die Bedingung irgendwann **False** wird — sonst bekommst du eine Endlosschleife. Abbrechen mit **Ctrl + C** im Terminal.

4.4 · while mit Input-Validierung

Klassisches Pattern aus Serie 3: frage so lange, bis eine gültige Eingabe kommt.

```
zahl = int(input("Zahl zwischen 1 und 10: "))
while zahl < 1 or zahl > 10:
    print("Ungültig, nochmal.")
    zahl = int(input("Zahl zwischen 1 und 10: "))
print("Danke:", zahl)
```

4.5 · break und continue

break verlässt die Schleife sofort. **continue** springt zum nächsten Durchlauf.

```
# erste Zahl in der Liste finden, die durch 7 teilbar ist
for x in zahlen:
    if x % 7 == 0:
        print("Gefunden:", x)
        break

# alle ungeraden überspringen
for x in range(10):
    if x % 2 == 1:
        continue
    print(x)    # 0, 2, 4, 6, 8
```

5 · Listen & Strings

5.1 · Listen erstellen

```
leer      = []
zahlen    = [3, 1, 4, 1, 5, 9, 2, 6]
gemischt  = [1, "Hallo", 3.14, True]
nullen    = [0] * 5          # [0, 0, 0, 0, 0]
```

5.2 · Indexing (Zugriff auf Elemente)

Python zählt ab **0**. Negative Indizes zählen von hinten.

```
namen = ["Anna", "Ben", "Clara", "David"]
#      0      1      2      3
#      -4     -3     -2     -1

namen[0]    # "Anna"
namen[2]    # "Clara"
namen[-1]   # "David"   (letztes Element)
namen[-2]   # "Clara"
```

● Achtung

namen[4] würde einen **IndexError** werfen — der letzte Index ist **len(namen) - 1**.

5.3 · Slicing (Bereiche)

```
zahlen = [10, 20, 30, 40, 50]

zahlen[1:4]    # [20, 30, 40]    (Index 1 bis 3, 4 ist exklusiv)
zahlen[:3]     # [10, 20, 30]    (von Anfang bis Index 2)
zahlen[2:]     # [30, 40, 50]    (von Index 2 bis Ende)
zahlen[:]      # ganze Kopie der Liste
zahlen[::-1]   # [50, 40, 30, 20, 10] (umgedreht)
```

5.4 · Listen verändern

```
liste = [1, 2, 3]

liste.append(4)    # [1, 2, 3, 4]    ans Ende
liste.insert(0, 99) # [99, 1, 2, 3, 4] an Position einfügen
liste.remove(2)    # [99, 1, 3, 4]    entfernt erstes Vorkommen
liste.pop()        # entfernt und returniert das letzte
liste.pop(0)       # entfernt und returniert das erste
liste[1] = 77      # ersetzt Element an Index 1
```

5.5 · Nützliche Listen-Funktionen

```
len(liste)          # Anzahl Elemente
sum(zahlen)         # Summe
max(zahlen)         # größtes Element
min(zahlen)         # kleinstes Element
sorted(zahlen)      # sortierte Kopie
zahlen.sort()       # sortiert die Liste selbst (in place)
zahlen.reverse()    # dreht um (in place)
42 in zahlen        # True, wenn 42 enthalten ist
```

5.6 · Strings funktionieren fast wie Listen

```
s = "Python"

s[0]          # "P"
s[-1]         # "n"
s[1:4]        # "yth"
len(s)        # 6
"th" in s     # True
```

Aber: Strings sind **unveränderlich** (immutable). `s[0] = "X"` gibt einen Fehler.

Wichtige String-Methoden:

```
s.upper()        # "PYTHON"
s.lower()        # "python"
s.replace("P", "J") # "Jython"
"a,b,c".split(",") # ["a", "b", "c"]
", ".join(["a", "b"]) # "a, b"
" hallo ".strip() # "hallo" (Leerzeichen weg)
```

5.7 · f-Strings (formatierte Ausgabe)

Der eleganteste Weg, Variablen in Strings einzubauen. Ein **f** vor dem Anführungszeichen, geschweifte Klammern um den Ausdruck.

```
name = "Hani"
alter = 23
print(f"Hi {name}, du bist {alter} Jahre alt.")
print(f"In 5 Jahren: {alter + 5}")
print(f"Pi: {3.14159:.2f}") # mit 2 Nachkommastellen -> "3.14"
```

6 · Funktionen

Eine Funktion bündelt wiederverwendbaren Code unter einem Namen. Definition mit **def**, Wert zurückgeben mit **return**.

6.1 · Basis-Struktur

```
def funktionsname(parameter1, parameter2):  
    # Code  
    return ergebnis  
  
# Aufruf:  
x = funktionsname(wert1, wert2)
```

6.2 · Beispiele

```
def quadrat(x):  
    return x * x  
  
print(quadrat(5))      # 25  
print(quadrat(-3))     # 9
```

```
def begruesse(name, hoeftlich=True):  
    if hoeftlich:  
        return f"Guten Tag, {name}"  
    else:  
        return f"Hi {name}"  
  
begruesse("Anna")      # "Guten Tag, Anna"  
begruesse("Ben", hoeftlich=False) # "Hi Ben"
```

● Tipp

hoeftlich=True ist ein **Default-Parameter**. Wenn der Aufrufer nichts angibt, wird der Standardwert verwendet.

6.3 · return vs. print

Das ist die häufigste Verwirrung für Anfänger:

- **print()** zeigt etwas auf dem Bildschirm an.
- **return** gibt einen Wert an die aufrufende Stelle zurück.

```
def addiere(a, b):
    return a + b

x = addiere(3, 4)    # x ist jetzt 7
print(x)             # 7
print(addiere(1, 1)) # 2 (gleichzeitig returnen und printen möglich)
```

● **Achtung**

Eine Funktion **ohne return** returt automatisch **None**. Das ist oft *nicht*, was du willst.

6.4 · Mehrere Rückgabewerte

```
def min_max(liste):
    return min(liste), max(liste)

a, b = min_max([3, 1, 4, 1, 5, 9])
print(a, b)    # 1 9
```

6.5 · Lokale vs. globale Variablen

Variablen, die **innerhalb** einer Funktion definiert werden, existieren nur dort ("lokal"). Außerhalb sind sie unbekannt.

```
def test():
    x = 10          # lokal
    print(x)

test()             # 10
# print(x)         würde NameError geben
```

7 · Module: math & random

Ein **Modul** ist eine Sammlung von Funktionen und Konstanten, die du **importieren** kannst. Python hat viele Module dabei — du brauchst sie nicht zu installieren.

7.1 · Das math-Modul

```
import math

math.pi          # 3.141592653589793
math.e           # 2.718281828459045

math.sqrt(16)     # 4.0           Wurzel
math.floor(3.7)   # 3            abrunden
math.ceil(3.2)    # 4            aufrunden
math.factorial(5) # 120          5! = 1·2·3·4·5
math.gcd(12, 18)  # 6            größter gem. Teiler
math.log(8, 2)    # 3.0          log2(8)
math.sin(math.pi/2) # 1.0
math.cos(0)       # 1.0
```

● Tipp

Alternative Import-Form: `from math import sqrt, pi` — dann schreibst du einfach `sqrt(16)` statt `math.sqrt(16)`.

7.2 · Das random-Modul

Für Zufallszahlen. Aus Serie 3.

```
import random

random.random()      # float zwischen 0.0 und 1.0
random.randint(1, 6) # int zwischen 1 und 6 (beide inklusiv!)
random.randrange(0, 10) # int zwischen 0 und 9 (stop exklusiv)
random.choice([1, 2, 3]) # ein zufälliges Element
random.shuffle(liste)  # mischt die Liste (in place)
```

● Achtung

random.randint(a, b) ist **inklusiv** auf beiden Seiten — anders als **range()**. Das ist eine häufige Fehlerquelle.

Reproduzierbare Zufallszahlen (zum Debuggen):

```
random.seed(42)      # gleicher Seed = gleiche Zahlenfolge
print(random.randint(1, 100))
```

8 · Typische Patterns aus Serien 1–5

8.1 · Summe / Produkt akkumulieren

```
# Summe der ersten 100 Zahlen
summe = 0
for i in range(1, 101):
    summe += i          # kurz für: summe = summe + i

# Produkt 1·2·3·...·n (Fakultät zu Fuß)
produkt = 1
for i in range(1, n + 1):
    produkt *= i
```

8.2 · Zählen, wie oft etwas passiert

```
# Wie viele Zahlen in der Liste sind gerade?
anzahl = 0
for x in zahlen:
    if x % 2 == 0:
        anzahl += 1
```

8.3 · Minimum / Maximum selbst finden

```
# ohne min() / max()
groesster = zahlen[0]
for x in zahlen[1:]:
    if x > groesster:
        groesster = x
```

8.4 · Liste aufbauen

```
# alle Quadrate von 1 bis 10
quadrade = []
for i in range(1, 11):
    quadrade.append(i ** 2)

# das gleiche als List Comprehension (kürzer, fortgeschritten)
quadrade = [i ** 2 for i in range(1, 11)]
```

8.5 · Input-Schleife mit Abbruch

```

summe = 0
while True:
    eingabe = input("Zahl (q zum Beenden): ")
    if eingabe == "q":
        break
    summe += int(eingabe)
print("Gesamt:", summe)

```

8.6 · Ist die Zahl prim?

```

def ist_prim(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

print(ist_prim(7))    # True
print(ist_prim(15))   # False

```

● Tipp

Effizienter: es reicht, bis `math.sqrt(n)` zu testen. Denn wenn n einen Teiler größer als $\sqrt{n}$ hat, dann hat es auch einen kleineren.

8.7 · Zahl umdrehen

```

# über String
n = 12345
umgedreht = int(str(n)[::-1])    # 54321

# mathematisch (ohne String)
n, ergebnis = 12345, 0
while n > 0:
    ergebnis = ergebnis * 10 + n % 10
    n //= 10

```


9 · Fehler lesen & Debugging

Python-Fehler sehen einschüchternd aus, folgen aber immer dem gleichen Aufbau. Die **letzte Zeile** sagt dir, was schiefging. Die Zeile darüber sagt dir, wo.

9.1 · Die häufigsten Fehler

Fehler	Bedeutung	Typische Ursache
SyntaxError	Python versteht deinen Code nicht.	Vergessener Doppelpunkt, fehlende Klammer, falscher Indent.
NameError	Diese Variable gibt es nicht.	Tippfehler im Namen, oder Variable wurde nie definiert.
TypeError	Falscher Typ für diese Operation.	"3" + 5 geht nicht. Einer ist str, einer int.
ValueError	Richtiger Typ, aber ungültiger Wert.	int("abc") — "abc" ist kein gültiger int-Text.
IndexError	Index außerhalb der Liste.	liste[10] bei einer Liste mit 5 Elementen.
ZeroDivisionError	Division durch 0.	x / 0 oder x % 0.
IndentationError	Falsche Einrückung.	Leerzeichen und Tabs gemischt, oder unerwarteter Indent.

9.2 · Debugging-Strategie

- **print() ist dein Freund.** Setz überall `print(variable)` rein, um zu sehen, was die Variable zu dem Zeitpunkt enthält.
- **Isolieren.** Wenn es nicht funktioniert, kommentier Zeilen aus, bis es geht. Dann weißt du, wo der Fehler sitzt.
- **Typen prüfen.** `print(type(x))` zeigt, ob x ein String oder eine Zahl ist.
- **Einrückung genau prüfen.** 4 Leerzeichen, immer. Nicht Tabs, nicht gemischt.

● Tipp

Wenn du einen Fehler bekommst, den du nicht verstehst: **kopiere die letzte Zeile der Fehlermeldung** und such danach. Jemand hatte diesen Fehler schon vor dir.

— Ende —

Viel Erfolg bei EPROG, Hani. Wenn etwas im Cheatsheet fehlt oder du eine Serie tiefer durchgehen willst, einfach Bescheid sagen.